



nextwork.org

Build a RAG API with FastAPI



Tiham Shafi

The screenshot displays a REST client interface for a POST request to the endpoint `/query`. The request body is a JSON object with a single key `q` (required) containing the string `What is kubernetes?`. The response is a 200 status code with a JSON body containing an `answer` field. The response headers show `content-length: 349`, `content-type: application/json`, `date: Wed, 14 Jun 2023 13:37:34 GMT`, and `server: uvicorn`.

Parameters

Name	Description
q * required string (query)	What is kubernetes?

Execute **Clear**

Responses

Code **Details**

200

Response body

```
{
  "answer": "Kubernetes is a container orchestration platform that enables organizations to automate the deployment, scaling, and management of containerized applications. It offers several benefits including improved efficiency, scalability, and agility, allowing businesses to quickly deploy and manage their applications in multiple environments."
}
```

Response headers

```
content-length: 349
content-type: application/json
date: Wed, 14 Jun 2023 13:37:34 GMT
server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------



Introducing Today's Project!

In this project, I will demonstrate how to build RAG API from the scratch. We are doing this project to learn how RAG works, Ollama works, and how to build an API and run it locally.

Key services and concepts

Services I used were Fastapi and Ollama. Key concepts we learnt includes virtual environment, embedding, endpoints and swagger UI interactions.

Challenges and wins

This project took me approximately 1 hour to get each and every concepts. The most challenging part was Python installation because at this time Python's 3.14.x versions doesn't contain all the services. On the other hand, It's most rewarding part was to get to know about RAG and FastApi.

Why I did this project

The project both contents AI concepts and Python based application indications. It's just a stepping stone. Long way to go :)



Setting Up Python and Ollama

In this step, I'm setting up Python and Ollama. Python is a programming language, as we are building RAG API. Ollama helps to run LLMs locally. I need these tools because our RAG API needs both equally - we need Python for the API code to work, and we need LLMs in order to transform the raw data in our knowledge base into personalised responses.

```
PS C:\WINDOWS\System32> ollama run tinyllama
>>> What is Kubernetes?
Kubernetes is an ancient Greek word that means "the great of the city" or "city's greatest". It's also used as a prefix for more specific names like Kubernetes, which means "the chief of the citizens" or "citizen chief", and Kerberos, which is the name of the fierce dog in Greek mythology. In English, it may be referred to simply as "Kuber".
```

Ollama and tinyllama ready

Ollama is tool to install and run LLMs locally. I downloaded the tinyllama model because it is lightweight and works for this project perfectly. The model will help my RAG API by converting raw chunks of data in our knowledge base into insightful answers.



Setting Up a Python Workspace

In this step, I'm setting up a Python workspace. We need it because it prevents package version conflicts between different projects.

Virtual environment

A virtual environment isolates a workspace from other projects. I created one for this project to isolate the dependencies from other Python based projects, so that it won't cause any conflicts. Once I activate it, it took me in a virtual environment, where I will do my work mainly related to the project. To create a virtual environment, I needed to run venv, which stores all the dependencies and scripts. And based on that my Python project will run.

Dependencies

The packages I installed are uvicorn, chromadb, fastapi and ollama. FastAPI is used as the web framework here to call the endpoints. Chroma is used for storing embeded docs to enhance the knowledge base. Uvicorn is the ASGI web server to help Python app connect and communicate with the outer world. And Ollama is used for running the LLM model locally.



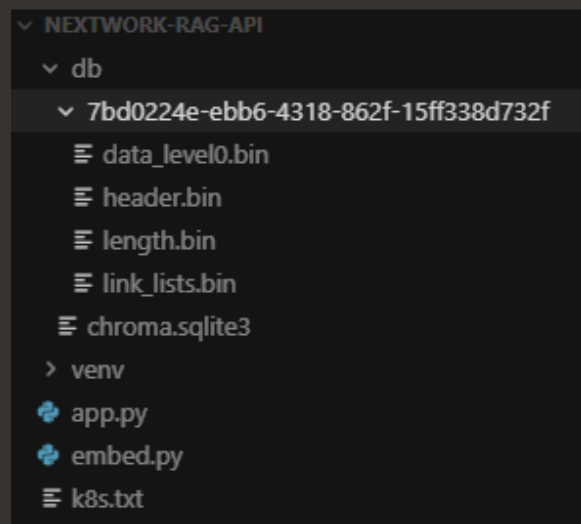
```
(venv) PS D:\STUDY\projects\network\network-rag-api> pip list | Select-String "fastapi|uvicorn|chromadb|ollama"

chromadb          1.4.0
fastapi           0.128.0
ollama            0.6.1
uvicorn           0.40.0
```



Setting Up a Knowledge Base

In this step, we are going to build a RAG API. We are creating files like technical questions to build a knowledge base about DevOps concepts. We need it to get more accurate answers. So with the API endpoint we will communicate with our API - requests will go through endpoints to retrieve data from our knowledge base.



Embeddings created

Embeddings are numerical representations (vectors) of real-world objects. We generated some texts which we were storing in a vector database (Chroma). The db/ folder contains some .bin folders after embedding with a sql file. This is important for RAG because RAG relies on retrieving relevant context from a knowledge base (like your Chroma vector database) using these embeddings.



Building the RAG API

In this step, I'm building a RAG API. An API is a communication method to get requests and give responses accordingly. FastAPI is web framework for Python, which takes care of a lot of this background work like, it handles how requests come in and go out, checks if incoming information is correct, and returns answers in the right format.

How the RAG API works

My RAG API works by combining a language model's generative power with an external, up-to-date knowledge base to provide accurate and context-aware responses.

```
o (venv) PS D:\STUDY\projects\network\network-rag-api> uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['D:\STUDY\projects\network\network-rag-api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [13188] using WatchFiles
INFO: Started server process [10064]
INFO: Waiting for application startup.
INFO: Application startup complete.
[]
```



Testing the RAG API

In this step, testing my RAG API. Swagger UI gave a nice interface to interact with the endpoints directly without manually typing the command lines.

API query breakdown

I queried my API by running the command. The command uses the POST method, which means it is post requesting our query to get a response. The API responded with the answer we expected which was embedded by us.



The screenshot displays the Swagger UI interface for a POST endpoint at `/query`. The 'Parameters' section shows a required query parameter `q` with the value `What is kubernetes?`. Below this, the 'Execute' button is highlighted. The 'Responses' section shows a successful 200 status code. The 'Response body' is a JSON object containing a description of Kubernetes. The 'Response headers' section shows the following headers: `content-length: 349`, `content-type: application/json`, `date: Wed, 14 Jan 2020 13:37:34 GMT`, and `server: nginx`.

```
POST /query Query
```

Parameters

Name	Description
q * required string (query)	What is kubernetes?

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/query?q=What%20is%20kubernetes%3F' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/query?q=What%20is%20kubernetes%3F
```

Server response

Code	Details
200	Response body <pre>{ "answer": "Kubernetes is a container orchestration platform that enables organizations to automate the deployment, scaling, and management of containerized applications. It offers several benefits including improved efficiency, scalability, and agility, allowing businesses to quickly deploy and manage their applications in multiple environments." }</pre> Response headers <pre>content-length: 349 content-type: application/json date: Wed, 14 Jan 2020 13:37:34 GMT server: nginx</pre>

Responses

Code	Description	Links
------	-------------	-------

Swagger UI exploration

Swagger UI is a web UI which allows developers to visualize, understand, and test RESTful APIs without writing code. We used it to test our POST API by going to `http://127.0.0.1:8000/docs` instead of writing the code. The best part about using Swagger UI was its visualization to understand the endpoints and testing it out.



Adding Dynamic Content

In this project extension, we are upgrading our RAG API by creating a new endpoint that makes it easy to update our knowledge base. /add will let us add new info through an API endpoint instead of uploading a new text file into our project's /db folder. It also saves us the effort of re-running the embeddings script.

POST /add Add Knowledge

Add new content to the knowledge base dynamically.

Parameters

Name	Description
text required	RAG combines your own document search w

string (query)

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/add?text=RAG%20combines%20your%20own%20document%20search%20with%20AI%20generation%20to%20get%20relevant%20information%20for%20better%20accuracy.' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/add?text=RAG%20combines%20your%20own%20document%20search%20with%20AI%20generation%20to%20get%20relevant%20information%20for%20better%20accuracy.
```

Server response

Code	Details
200	Response body <pre>{ "status": "success", "message": "Content added to knowledge base", "id": "9ba263bf-422e-4659-b5ec-fb93afcb1ecc" }</pre> Response headers <pre>content-length: 188 content-type: application/json date: Wed, 14 Jan 2026 14:25:49 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Dynamic content endpoint working

The /add endpoint allows to dynamically add new content into our knowledge base. Dynamic content means content that can be updated, added, deleted without needing to stop our application. In our case, because content can be added via /add endpoint, we no longer need to manually stop our application.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

